ASSESSMENT OBJECT DETECTION · DOKUMENTASI FINAL

Laporan Teknis Lengkap PPE Vision

Panduan memahami seluruh hubungan kode, model YOLO, Laravel, Python, Streamlit, MySQL, Ollama, deployment, keamanan, dan evaluasi model—disusun untuk belajar serta persiapan presentasi/rekrutmen.

3 kelas	69,3%	14 / 75
Helm · Masker · People	mAP50-95 test independen	test lulus / assertions

Proyek: PPE Vision
Repository: github.com/silviadela/ppe-vision

Versi audit: commit f7247b9
Tanggal: 15 Oktober 2026

PANDUAN MEMBACA

Cara menggunakan laporan ini

Laporan ini dibuat dalam urutan “gambaran besar → alur → kode → deployment → pertanyaan wawancara”. Bila waktu belajar terbatas, gunakan jalur cepat berikut.

15 menit

Baca Bab 1, 2, 6, 9, dan ringkasan presentasi pada Bab 18.

60 menit

Tambahkan Bab 4, 7, 8, 10, 12, dan 13 untuk memahami implementasi.

Mendalam

Baca semuanya sambil membuka file yang dicantumkan pada setiap bagian.

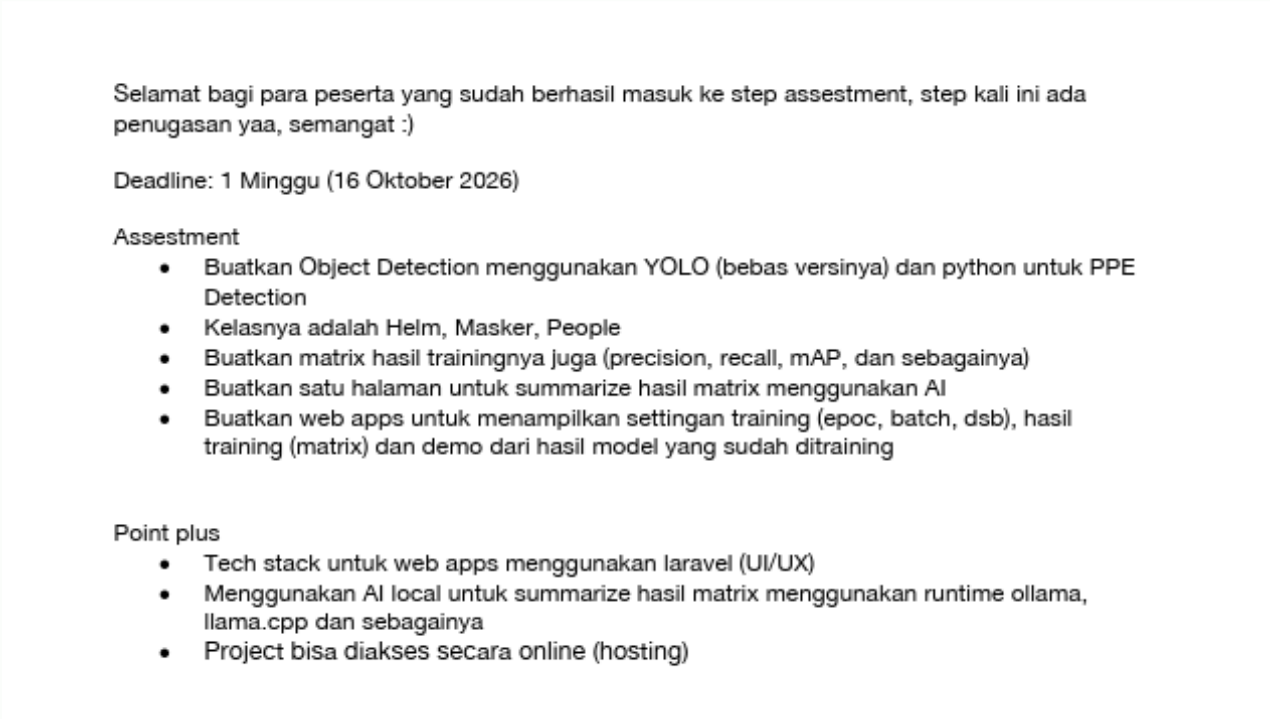
Prinsip utama proyek: Laravel adalah portal utama; Python menjalankan YOLO; Streamlit menyediakan demo online; MySQL menyimpan data; Ollama menghasilkan penjelasan lokal; seluruh angka metrik tetap berasal dari database agar AI tidak “mengarang angka”.

Daftar isi

1. Pemetaan syarat assessment
2. Arsitektur sistem
3. Teknologi dan versi
4. Peta folder dan tanggung jawab
5. Dataset, training, dan pemilihan model
6. Metrik dan interpretasi hasil
7. Alur inferensi YOLO
8. Cara kerja Laravel
9. Integrasi Ollama dan kontrol halusinasi
10. Database dan hubungan data
11. UI/UX dan halaman aplikasi
12. Deployment online
13. Keamanan
14. Pengujian dan audit
15. File aktif, pendukung, dan opsional
16. Menjalankan sistem lokal
17. Troubleshooting
18. Naskah presentasi
19. Pertanyaan wawancara dan jawaban
20. Glosarium dan link final

BAB 1

Pemetaan syarat assessment



Selamat bagi para peserta yang sudah berhasil masuk ke step assestment, step kali ini ada penugasan yaa, semangat :)

Deadline: 1 Minggu (16 Oktober 2026)

Assestment

- Buatkan Object Detection menggunakan YOLO (bebas versinya) dan python untuk PPE Detection
- Kelasnya adalah Helm, Masker, People
- Buatkan matrix hasil trainingnya juga (precision, recall, mAP, dan sebagainya)
- Buatkan satu halaman untuk summarize hasil matrix menggunakan AI
- Buatkan web apps untuk menampilkan settingan training (epoc, batch, dsb), hasil training (matrix) dan demo dari hasil model yang sudah ditraining

Point plus

- Tech stack untuk web apps menggunakan laravel (UI/UX)
- Menggunakan AI local untuk summarize hasil matrix menggunakan runtime ollama, llama.cpp dan sebagainya
- Project bisa diakses secara online (hosting)

Gambar syarat assessment yang menjadi dasar audit implementasi.

Syarat	Status	Bukti implementasi	Penjelasan singkat
Object Detection memakai YOLO dan Python	Terpenuhi	`inference_service/main.py`, `streamlit_app/app.py`, `best.pt`	Ultralytics YOLO dipanggil dari Python untuk inferensi gambar, video, dan kamera.
Kelas Helm, Masker, People	Terpenuhi	`model-info.json`, `CLASS_ALIASES`	Kelas sumber *Helmet, Mask, Person* dipetakan menjadi istilah aplikasi *helm, masker, people*.
Matriks hasil training: precision, recall, mAP, dsb.	Terpenuhi	Dashboard, detail eksperimen, grafik training, confusion matrix, PR/F1/P/R curve	Angka utama berasal dari 212 gambar test independen; training curve berasal dari validation per epoch.
Satu halaman AI untuk merangkum hasil	Terpenuhi	`/experiments/1/summary`, `MetricSummaryService.php`	Ollama lokal menulis risiko dan rekomendasi; angka dihasilkan deterministik dari database.
Web app untuk setting training, hasil, dan demo	Terpenuhi	Menu Eksperimen, Dashboard, Demo Deteksi	Form tulis aktif pada lingkungan lokal; pada production dinonaktifkan untuk keamanan, tetapi hasil tetap dapat dilihat.
Point plus: Laravel UI/UX	Terpenuhi	Laravel 12, Blade, CSS responsif, Vite	Laravel menjadi website utama, bukan sekadar wrapper atau halaman tautan.

Syarat	Status	Bukti implementasi	Penjelasan singkat
Point plus: AI lokal Ollama/Llama	Terpenuhi	Ollama 0.40.4, `llama3.2:3b`	AI dijalankan lokal, hasil disimpan ke database, lalu dapat ditampilkan online tanpa membuka Ollama ke internet.
Point plus: bisa diakses online	Terpenuhi	InfinityFree + Streamlit Community Cloud	Portal, data, metrik, AI summary, dan demo tersedia melalui link publik.

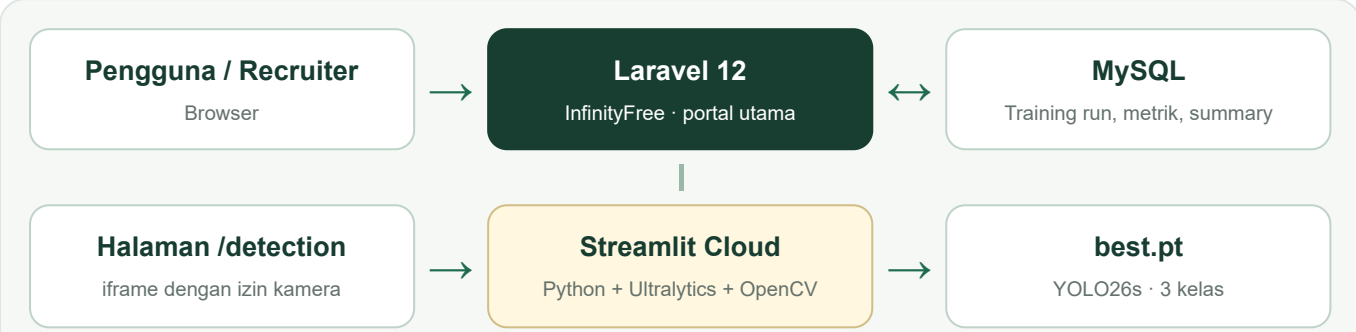
Kesimpulan kepatuhan: seluruh syarat inti dan tiga point plus telah diimplementasikan. Batasannya bukan syarat yang hilang, melainkan karakter hosting gratis: Streamlit dapat mengalami cold start dan Ollama lokal tidak dapat dipanggil langsung oleh server InfinityFree.

BAB 2

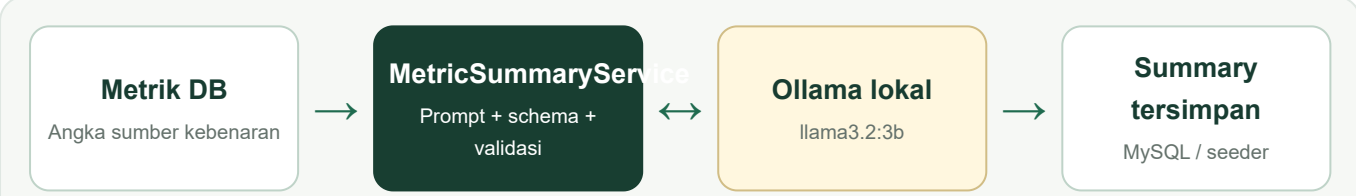
Arsitektur sistem: siapa terhubung ke siapa?

Sistem sengaja dipisah menjadi beberapa komponen karena InfinityFree hanya menjalankan PHP/MySQL, sedangkan YOLO memerlukan Python, PyTorch, dan resource inferensi.

Arsitektur production yang aktif

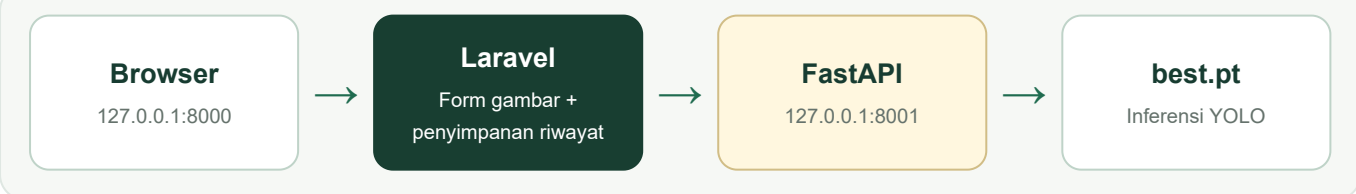


Arsitektur AI summary



Ollama berjalan pada komputer pengembang di `127.0.0.1:11434`. Server InfinityFree tidak dapat mengakses alamat lokal tersebut. Karena itu summary dibuat saat lokal, disimpan sebagai Markdown terstruktur, kemudian ditanam melalui seeder ke database production. Pendekatan ini tetap memenuhi “AI lokal” sekaligus membuat halaman online stabil.

Arsitektur pengembangan lokal alternatif



> **Kalimat mudah diingat:** “Laravel mengatur pengalaman pengguna dan data; Python menjalankan AI; Streamlit membawa AI Python ke internet; Ollama menjelaskan hasil secara lokal.”

BAB 3

Teknologi, versi, dan alasan pemilihan

Teknologi	Versi audit	Peran	Alasan
Laravel Framework	12.69.3	Portal, routing, validasi, DB, UI	Memenuhi point plus Laravel dan memberi struktur MVC yang jelas untuk dipresentasikan.
PHP	8.2.12 lokal; 8.4.25 hosting	Runtime Laravel	Kompatibel dengan requirement Composer `^8.2`.
MySQL	InfinityFree, port 3306	Database production persisten	Tersedia gratis di InfinityFree dan cocok dengan Eloquent/migration Laravel.
SQLite	Lokal/test	Database pengembangan	Setup cepat tanpa server database terpisah.
Python	3.12.10	Inferensi dan demo AI	Ekosistem utama Ultralytics, PyTorch, OpenCV, dan Streamlit.
Ultralytics	8.4.176	Memuat dan menjalankan checkpoint YOLO	API prediksi konsisten untuk gambar/video dan menyediakan speed serta bounding box.
PyTorch	2.14.1+cpu lokal	Backend model	Runtime neural network yang digunakan Ultralytics.
FastAPI	0.143.0	API inferensi lokal	Ringan, validasi request baik, dan mudah dihubungkan dengan Laravel melalui HTTP.
Streamlit	1.65.0	Demo online gambar/video/kamera	Dapat menjalankan Python pada hosting gratis dan mudah ditanam sebagai iframe.
Ollama	0.40.4	Runtime LLM lokal	Privat, tidak memerlukan API key, dan sesuai point plus AI lokal.
Llama	llama3.2:3b, ±2 GB	Menulis risiko/rekomendasi	Cukup ringan untuk laptop, mendukung Bahasa Indonesia, dan respons terstruktur.
Vite	6.x	Build CSS/JS	Menghasilkan asset production yang dapat diunggah tanpa Node.js di server.
FileZilla	Client FTP	Upload ke InfinityFree	InfinityFree gratis tidak menyediakan SSH/Composer/NPM.

Mengapa bukan satu server? Menjalankan Laravel, MySQL, PyTorch, model 20 MB, Ollama 2 GB, dan pemrosesan video pada satu free hosting tidak realistis. Pemisahan layanan adalah keputusan arsitektur berdasarkan kemampuan runtime, bukan duplikasi aplikasi.

BAB 4

Peta folder dan tanggung jawab

```
test_wingsgrup/
├── app/
│   ├── Http/Controllers/       # Menerima request dan memilih response
│   ├── Models/                 # TrainingRun dan Detection (Eloquent)
│   └── Services/               # Integrasi YOLO API dan Ollama
├── config/                     # Membaca variabel .env ke konfigurasi aplikasi
├── database/
│   ├── migrations/             # Struktur tabel
│   ├── seeders/                # Data model final dan summary
│   └── data/                   # Metrik JSON/CSV asli
├── inference_service/
│   ├── main.py                 # FastAPI lokal
│   └── models/best.pt          # Checkpoint final
├── streamlit_app/app.py        # Demo online gambar/video/kamera
├── resources/
│   ├── views/                  # Blade UI
│   └── css/app.css             # Desain responsif
├── public/training/            # Grafik dan artefak evaluasi
├── routes/web.php              # Daftar URL → controller
├── deploy/infinityfree/        # Build package + .htaccess + update-safe
├── tests/Feature/              # Pengujian end-to-end Laravel
└── docs/ppe-vision-report/     # Laporan ini
```

Hubungan file paling penting

Mulai dari	Terhubung ke	Tujuan
`routes/web.php`	Controller	Memetakan URL ke aksi aplikasi.
Controller	Model / Service / View	Mengatur alur request tanpa mencampur logika eksternal dengan HTML.
`TrainingRun`	`training_runs`	Menyimpan konfigurasi, metrik, artefak, history, summary.
`MetricSummaryService`	Ollama API	Mengirim prompt terstruktur dan memvalidasi jawaban.
`YoloInferenceService`	FastAPI `/health` dan `/predict`	Integrasi inferensi lokal.
`DetectionController@index`	Streamlit URL	Membuat URL `?embed=true` untuk iframe online.
`DatabaseSeeder`	JSON/CSV + MySQL	Menempatkan model final dan hasil audit ke database production.

BAB 5

Dataset, training, dan pemilihan model final

3.025	603	212
gambar train	gambar validation	gambar test independen

Model final

Nama eksperimen	PPE YOLO26s Dataset 4 Final	**Arsitektur**	YOLO26s
Dataset	PPE Detection and Compliance v5	**Epoch**	60 (best epoch 58)
Image size	640 px	**Batch**	16
Optimizer	AdamW	**lr0**	0,0005
Momentum	0,9	**Weight decay**	0,0001
Checkpoint	`inference_service/models/best.pt` · 20.316.805 byte · SHA-256 `c852c3a7…b12a0e9f`		

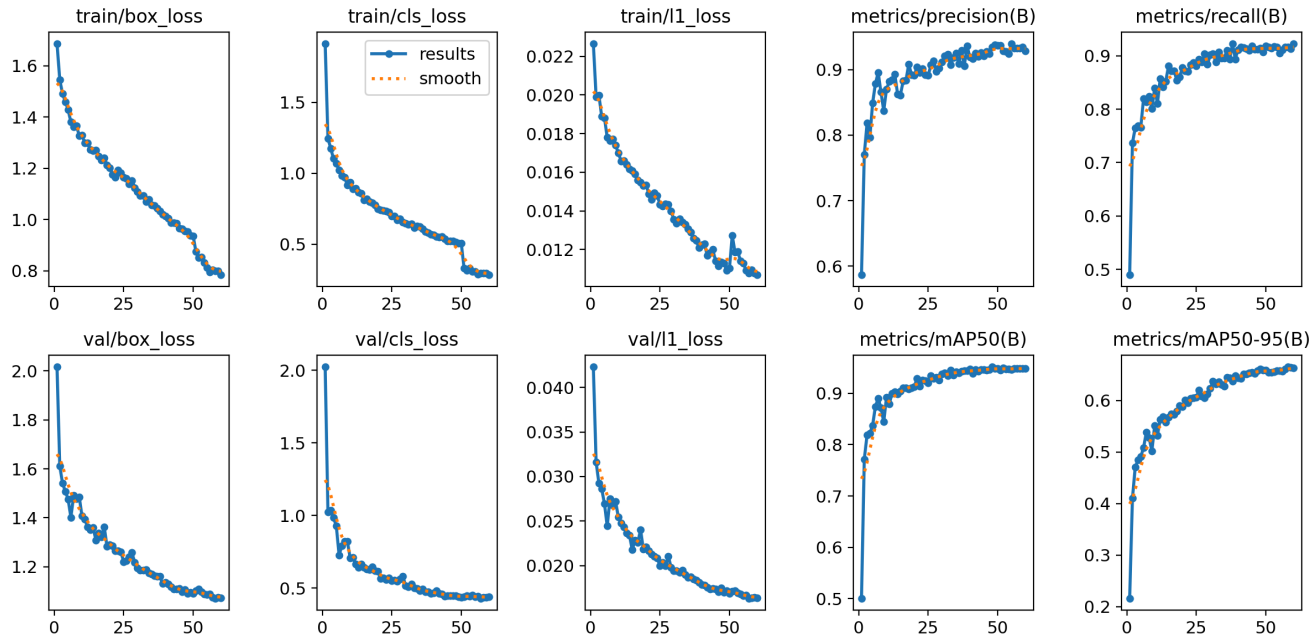
Mengapa menggunakan random search?

Hyperparameter seperti learning rate, momentum, dan weight decay memengaruhi cara model belajar. Lima trial pendek dijalankan selama 20 epoch pada 60% data. Trial 4 memiliki skor gabungan tertinggi pada catatan eksperimen, sehingga konfigurasinya dipakai untuk final training 60 epoch pada seluruh data train.

Trial	lr0	Momentum	Weight decay	Precision	Recall	mAP50-95	Skor
1	0,0005	0,95	0,0001	83,8%	73,8%	50,8%	0,6823
2	0,0005	0,90	0,0010	80,2%	75,1%	51,5%	0,6822
3	0,0020	0,95	0,0001	82,5%	72,7%	47,6%	0,6626
4	**0,0005**	**0,90**	**0,0001**	**81,1%**	**76,5%**	**50,4%**	**0,6848**
5	0,0020	0,90	0,0010	80,3%	75,6%	48,3%	0,6722

Pemisahan data yang benar

- **Train:** digunakan untuk memperbarui bobot model.
- **Validation:** dipantau setiap epoch untuk melihat proses belajar dan memilih checkpoint.
- **Test:** hanya untuk evaluasi akhir. Karena tidak dipakai melatih/menyetel model, angkanya lebih jujur untuk generalisasi.



Kurva training/validation 60 epoch. Dashboard menandai bahwa kurva per epoch berasal dari validation, sementara kartu utama berasal dari test independen.

BAB 6

Metrik model dan cara menjelaskannya

95,5%
Precision test

90,7%
Recall test

93,0%
F1 test

95,7%
mAP50 test

69,3%
mAP50-95 test

Masker
kelas terlemah · 63,4%

Definisi sederhana

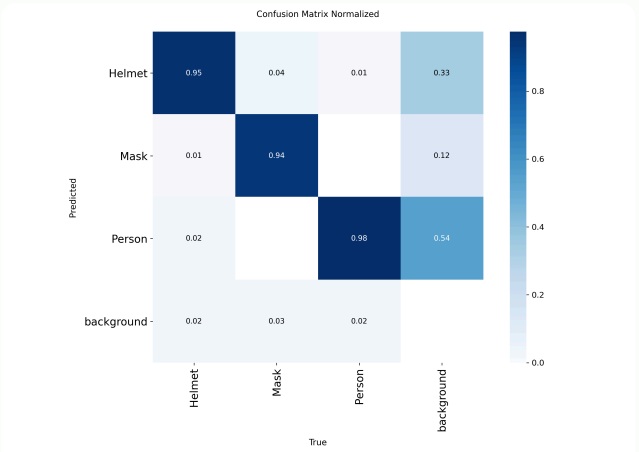
Metrik	Rumus / makna	Cara membaca hasil proyek
Precision	`TP / (TP + FP)`. Dari semua deteksi yang dibuat, berapa yang benar?	95,5% menunjukkan false positive relatif rendah.
Recall	`TP / (TP + FN)`. Dari semua objek yang seharusnya ditemukan, berapa yang berhasil ditemukan?	90,7% baik, tetapi masih ada risiko objek terlewat (false negative).
F1	`2 × P × R / (P + R)`. Keseimbangan precision dan recall.	93,0% menunjukkan keseimbangan kuat.
IoU	Luas irisan bounding box dibagi luas gabungan.	Mengukur ketepatan posisi kotak prediksi terhadap label.
AP	Luas di bawah precision–recall curve untuk satu kelas.	Meringkas performa kelas pada berbagai threshold.
mAP50	Rata-rata AP pada IoU 0,50.	95,7%; objek umumnya ditemukan dan kotaknya cukup bertumpuk.
mAP50-95	Rata-rata AP pada IoU 0,50 sampai 0,95.	69,3%; lebih ketat karena menilai ketepatan lokasi kotak pada banyak IoU.

Mengapa mAP50 tinggi tetapi mAP50-95 lebih rendah? mAP50 cukup puas jika kotak overlap 50%. mAP50-95 juga menguji overlap sangat ketat sampai 95%. Selisih ini berarti klasifikasi objek sudah kuat, tetapi posisi/ukuran bounding box masih dapat diperbaiki.

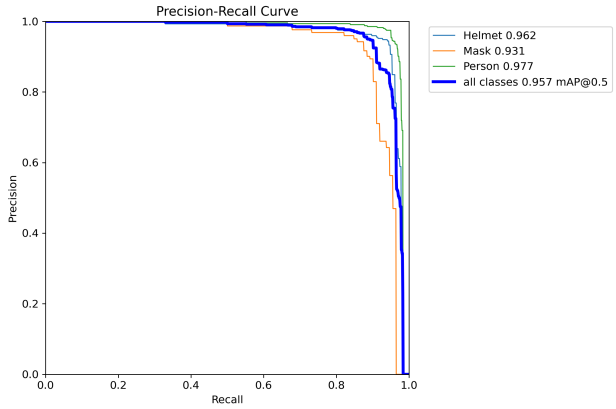
Per kelas pada test independen

Kelas aplikasi	Kelas model	Precision	Recall	F1	mAP50	mAP50-95
Helm	Helmet	96,3%	89,5%	92,8%	96,2%	69,1%
Masker	Mask	92,9%	87,5%	90,1%	93,1%	**63,4%**
People	Person	97,2%	95,1%	96,2%	97,7%	75,4%

Interpretasi: People paling stabil; Masker paling sulit karena ukuran relatif kecil, variasi bentuk/warna, occlusion pada wajah, sudut, cahaya, dan potensi label masker semuka. Perbaikan berikutnya harus fokus pada kualitas anotasi dan variasi data Masker.



Normalized confusion matrix untuk membaca kelas yang tertukar atau terlewat.



Precision–Recall curve menunjukkan trade-off precision dan recall pada berbagai confidence threshold.

BAB 7

Alur inferensi YOLO: dari input sampai bounding box

1. **Input**
   Pengguna memilih gambar, video, atau foto kamera di Streamlit.
2. **Validasi**
   Format, ukuran, dan nilai confidence diperiksa. FastAPI membatasi gambar 10 MB; Streamlit membatasi pilihan format UI.
3. **Load model**
   `best.pt` dimuat satu kali melalui cache agar request berikutnya lebih cepat.
4. **Predict**
   Ultralytics menjalankan `model.predict(..., conf=threshold, imgsz=image_size)`.
5. **Normalisasi kelas**
   `Helmet → helm`, `Mask → masker`, `Person → people`.
6. **Output**
   Sistem menggambar bounding box, label, confidence, membuat tabel, menghitung jumlah kelas, dan mencatat waktu inferensi.

```
CLASS_ALIASES = {
    "helmet": "helm",
    "mask": "masker",
    "person": "people",
}

result = model.predict(
    source=frame,
    conf=confidence,
    imgsz=image_size,
    verbose=False,
)[0]
```

Mengapa tiga kelas tidak selalu tampil sekaligus?

Model memiliki tiga kelas, tetapi output hanya berisi kelas yang benar-benar terdeteksi pada input dan melewati confidence threshold. Jika gambar hanya berisi orang dan helm, keluaran Masker bisa nol. Nilai nol bukan berarti kelas hilang dari model.

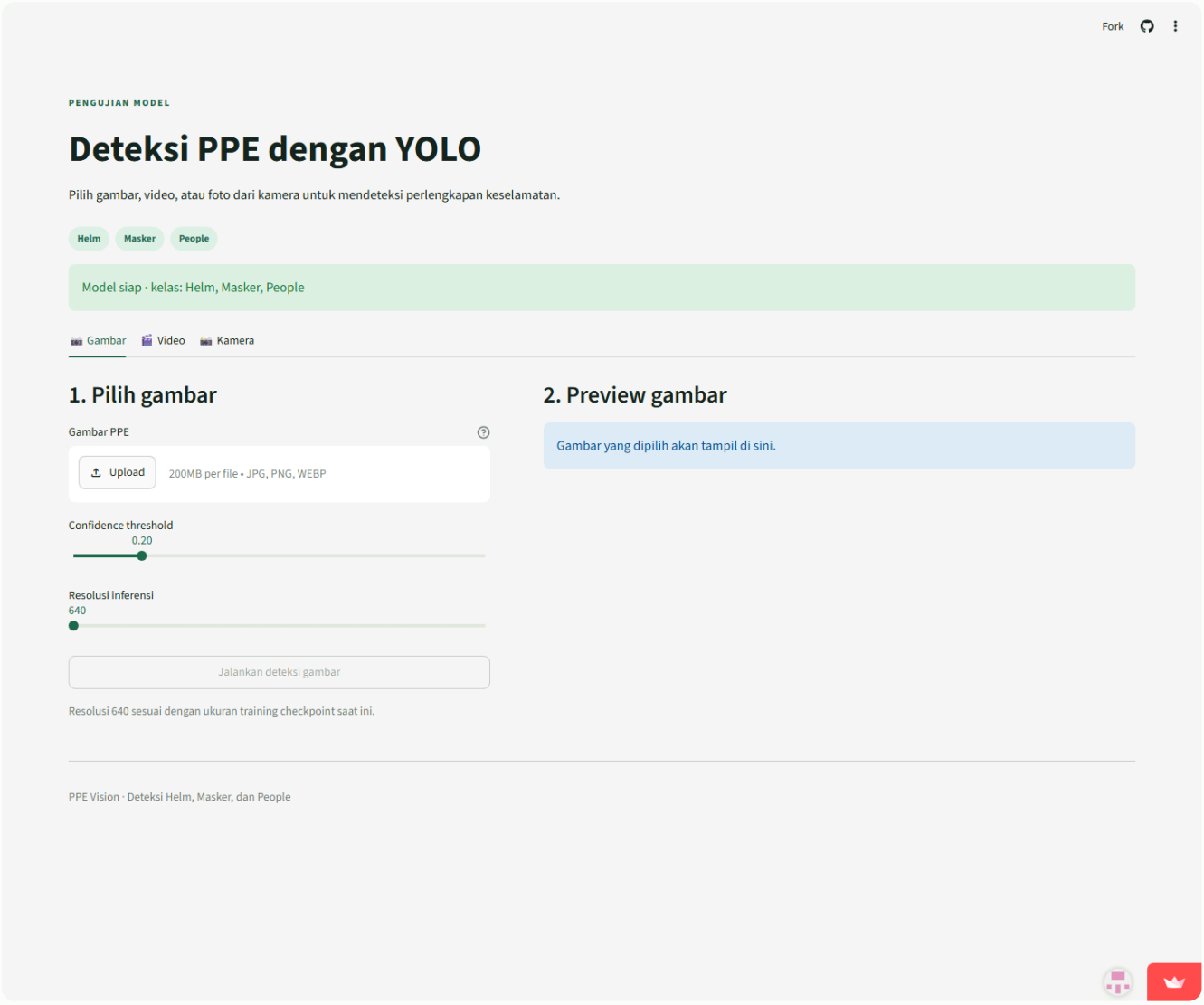
Confidence threshold

- Threshold rendah: lebih banyak objek ditemukan, tetapi false positive dapat meningkat.
- Threshold tinggi: prediksi lebih selektif, tetapi false negative dapat meningkat.
- Default proyek 0,20 dipilih agar objek PPE kecil tidak mudah terlewat; tetap perlu diuji pada kondisi lapangan.

Perbedaan gambar, video, dan kamera

Mode	Cara kerja	Catatan
Gambar	Satu frame → satu prediksi → bounding box + tabel.	Paling cepat untuk demo.

Mode	Cara kerja	Catatan
Video	OpenCV membaca frame; `frame_stride` dapat melewati sebagian frame; hasil dikodekan kembali.	FFmpeg mengubah output ke MP4; audio tidak disertakan.
Kamera	`st.camera_input` mengambil satu foto lalu diprediksi.	Tidak membutuhkan STUN/TURN karena bukan streaming real-time.

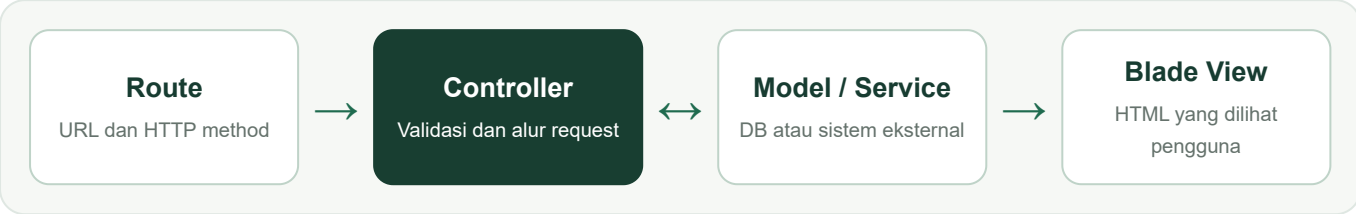


Demo online aktif: status model siap dan tiga mode input tersedia. Streamlit dapat tidur setelah tidak aktif, sehingga cold start 1–3 menit adalah karakteristik paket gratis.

BAB 8

Cara kerja Laravel dari route hingga tampilan

Pola MVC + Service



Daftar route penting

Method + URL	Controller	Fungsi
GET /	DashboardController@index	Menampilkan eksperimen terakhir, metrik, curve, dan per kelas.
GET /experiments	TrainingRunController@index	Daftar eksperimen.
GET /experiments/{id}	TrainingRunController@show	Konfigurasi dan artefak training.
POST /experiments/{id}/metrics	TrainingRunController@importMetrics	Impor metrik/CSV saat mode admin lokal aktif.
GET/POST /experiments/{id}/summary	SummaryController	Melihat atau menghasilkan AI summary.
GET/POST /detection	DetectionController	Embed Streamlit atau fallback FastAPI lokal.
GET/POST /install	InstallationController	Migrasi/seeding sekali pakai tanpa SSH.

Contoh koneksi Laravel ke konfigurasi layanan

```
// config/services.php
'yolo'      => ['url' => env('YOLO_API_URL')],
'streamlit' => ['url' => env('STREAMLIT_DEMO_URL')],
'ollama'    => [
    'enabled' => env('OLLAMA_ENABLED', false),
    'url'     => env('OLLAMA_URL', 'http://127.0.0.1:11434'),
    'model'   => env('OLLAMA_MODEL', 'llama3.2:3b'),
];
```

`.env` tidak dipanggil langsung di banyak tempat. Nilai dibaca oleh file config, lalu kode memanggil `config(...)`. Ini mengikuti praktik Laravel dan memudahkan cache konfigurasi.

Koneksi Laravel ke Streamlit

```
$streamlitUrl = filter_var(config('services.streamlit.url'), FILTER_VALIDATE_URL);
$streamlitEmbedUrl = $streamlitUrl . '?embed=true';

<iframe src="{{ $streamlitEmbedUrl }}"
        allow="camera; autoplay; fullscreen"></iframe>
```

Iframe membuat pengguna tetap berada pada layout Laravel. Atribut `allow="camera"` diperlukan agar tab Kamera Streamlit dapat meminta izin browser. Header CSP di root `.htaccess` hanya mengizinkan frame dari `*.streamlit.app`.

Koneksi Laravel ke FastAPI lokal

```
return Http::timeout(120)
    ->attach('file', fopen($file->getRealPath(), 'r'), $file->getClientOriginalName())
    ->post(rtrim($url, '/') . '/predict', ['confidence' => $confidence])
    ->throw()
    ->json();
```

`YoloInferenceService` memisahkan komunikasi HTTP dari controller. Controller fokus menyimpan file dan riwayat; service fokus berkomunikasi dengan Python.

BAB 9

Ollama: alasan, integrasi, dan kontrol halusinasi

Mengapa Ollama?

- **Lokal dan privat:** data metrik tidak perlu dikirim ke API pihak ketiga.
- **Tanpa API key/biaya per request:** sesuai assessment dan keterbatasan biaya.
- **Model ringan:** Llama 3.2 3B berukuran sekitar 2 GB dan dapat berjalan pada CPU.
- **Fallback tersedia:** bila Ollama mati atau timeout, sistem tetap menghasilkan ringkasan berbasis aturan.

Alur generasi

1. Laravel mengambil metrik dan per-class metrics dari `TrainingRun`.
2. Service menghitung kelas terlemah sendiri, bukan meminta LLM menebak.
3. Prompt mengirim data terstruktur, tetapi melarang LLM menulis angka/persentase.
4. Ollama harus mengembalikan JSON berisi `operational_risk` dan tepat tiga `recommendations`.
5. Laravel memvalidasi JSON, fokus kelas terlemah, kata “confusion” dan “confidence”. Jika gagal, fallback rule-based dipakai.
6. Angka kesimpulan dan per kelas disusun ulang langsung dari database, lalu Markdown disimpan.

```
$response = Http::connectTimeout(5)->timeout(180)->post(
    rtrim(config('services.ollama.url'), '/') . '/api/generate',
    [
        'model' => config('services.ollama.model'),
        'stream' => false,
        'prompt' => $this->prompt($run),
        'format' => $this->responseSchema(),
        'options' => ['temperature' => 0.2, 'num_predict' => 360],
    ]
);
```

Mengapa temperature 0,2?

Temperature rendah membuat output lebih konsisten dan tidak terlalu kreatif. Untuk analisis metrik, konsistensi lebih penting daripada variasi bahasa.

Bagaimana mencegah angka AI tidak sama dengan dashboard?

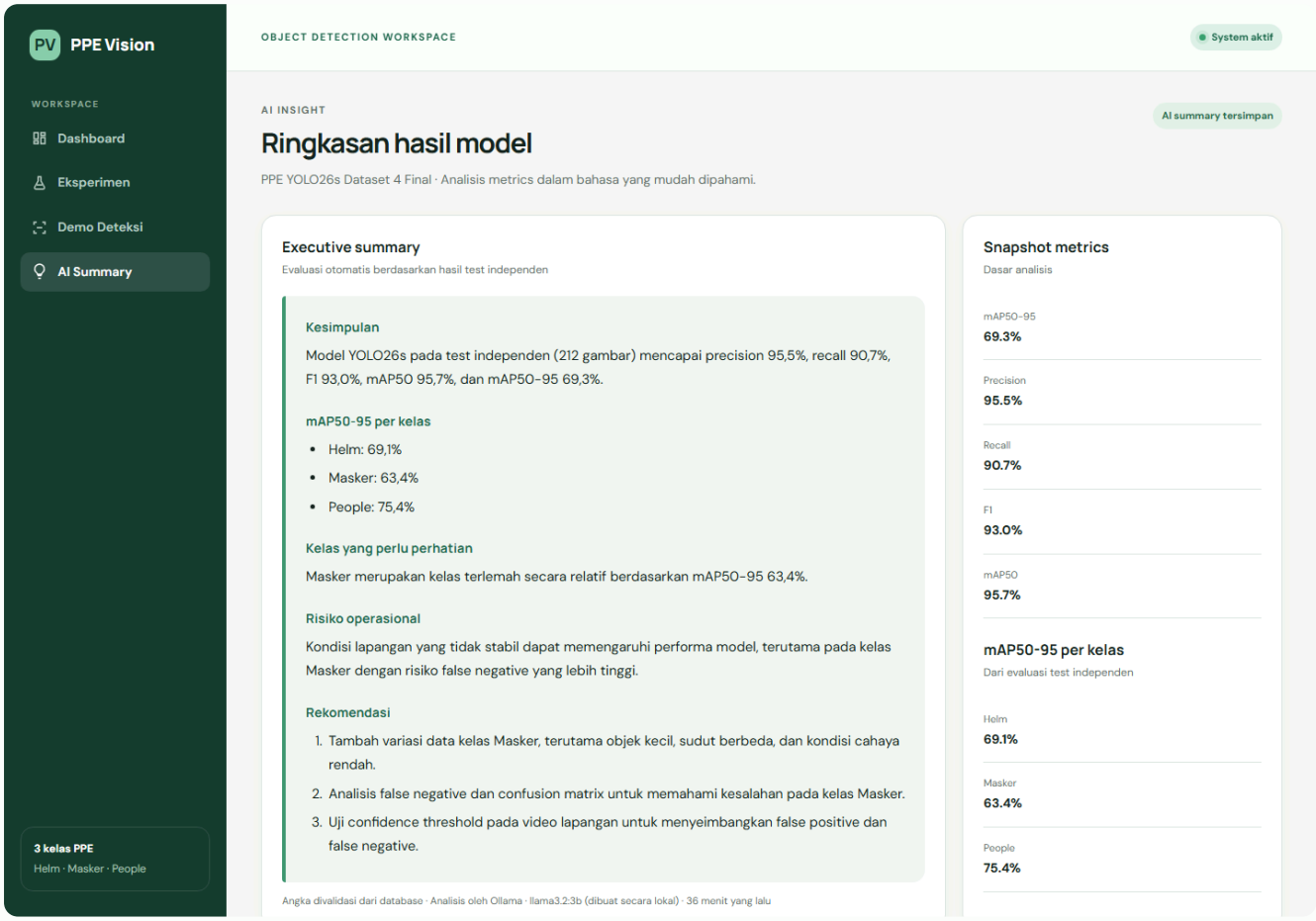
LLM tidak diberi tanggung jawab mencetak angka. Metode `structuredSummary()` mengambil precision, recall, F1, mAP50, mAP50-95, dan per-class mAP langsung dari database. Karena dashboard membaca record yang sama, keduanya konsisten.

> **Jawaban wawancara terbaik:** “Saya tidak menjadikan LLM sebagai sumber kebenaran numerik. Database adalah source of truth. LLM hanya membantu menyusun risiko operasional dan rekomendasi, lalu jawabannya divalidasi dan memiliki fallback deterministik.”

Konfigurasi lokal

```
OLLAMA_ENABLED=true
OLLAMA_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.2:3b
```

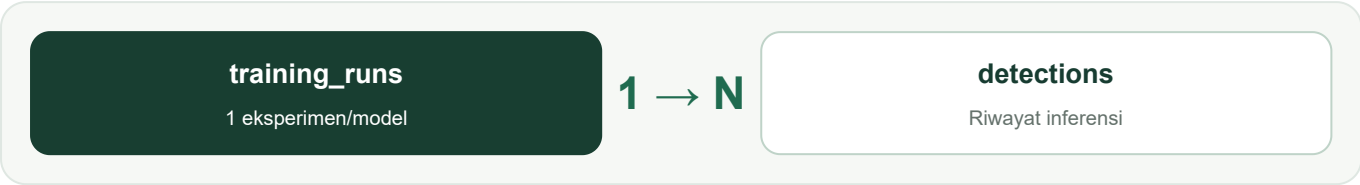
```
ollama serve
ollama pull llama3.2:3b
php artisan config:clear
```



AI Summary online. Angka divalidasi dari database; narasi tercatat dibuat oleh Ollama llama3.2:3b secara lokal.

BAB 10

Database dan hubungan data



Tabel `training_runs`

Menyimpan identitas model, konfigurasi training, status, timestamps, JSON metrik, metrik per kelas, confusion matrix, history per epoch, path artefak, AI summary, engine pembuat summary, dan catatan eksperimen.

Tabel `detections`

Menyimpan file sumber, output annotated image, array deteksi, waktu inferensi, mode, status, dan pesan error. Foreign key ke training run bersifat nullable dan menggunakan `nullOnDelete()`, sehingga riwayat tidak menyebabkan kegagalan jika eksperimen dihapus.

Mengapa kolom metrik memakai JSON?

Jumlah dan bentuk metrik eksperimen dapat berkembang. JSON memudahkan menyimpan nested validation, dataset split, dan artefak tanpa menambah kolom untuk setiap variasi. Identitas dan field yang sering difilter tetap memakai kolom biasa.

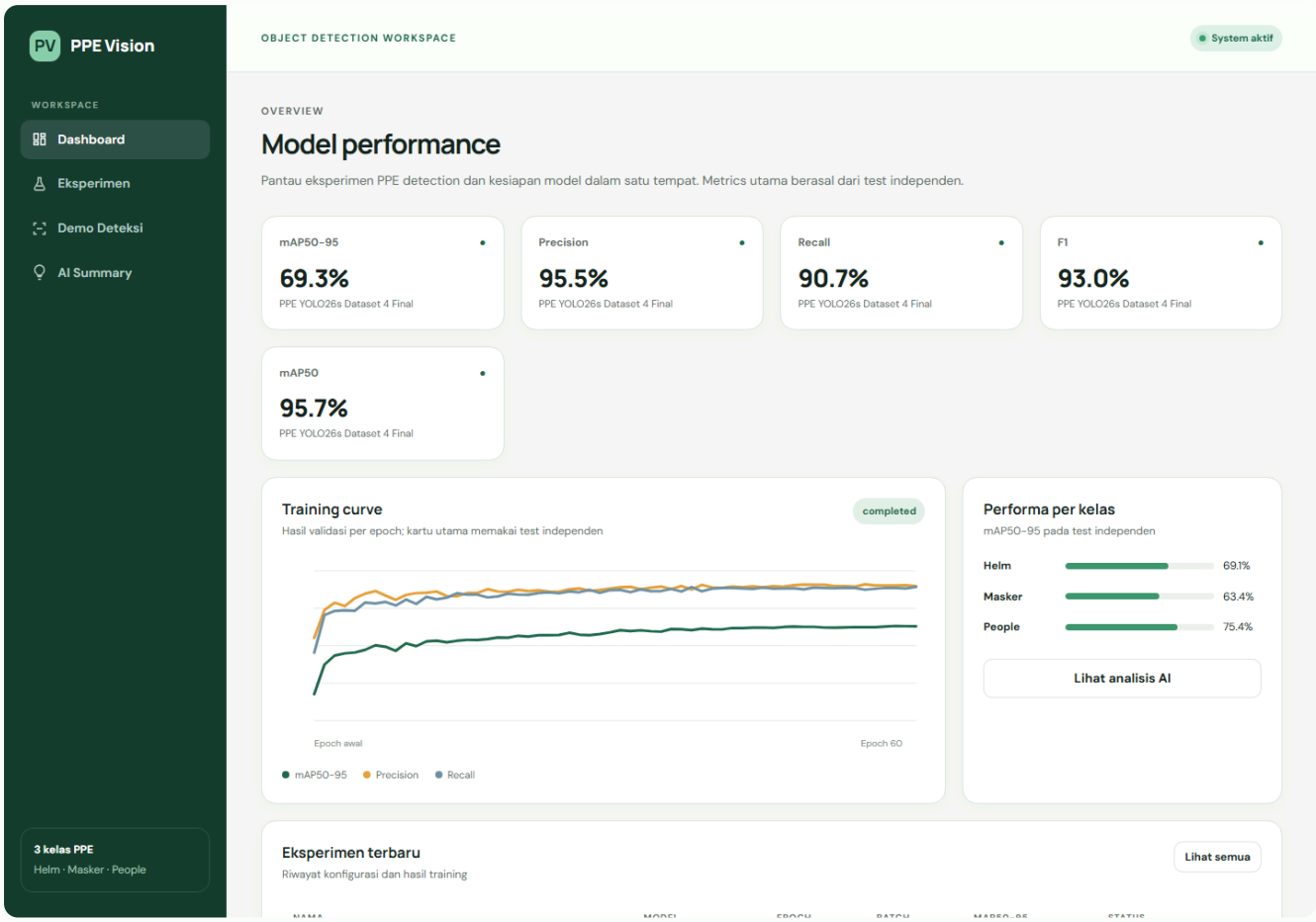
Seeder sebagai jembatan local → production

`DatabaseSeeder` mengisi satu record model final, membaca training history dari CSV, mencatat path artefak, serta menanam AI summary yang telah dibuat lokal. Di InfinityFree, seeder dijalankan melalui halaman installer karena tidak ada SSH.

> **Idempotensi praktis:** seeder mencari record melalui path checkpoint atau nama model lama, lalu memperbaruinya. Ini mencegah model final terus diduplikasi saat deployment diperbarui.

BAB 11

UI/UX dan halaman aplikasi



Dashboard production tervalidasi pada 15 Oktober 2026. Kartu utama memakai test independen dan kurva memakai validation per epoch.

Halaman	Tujuan pengguna	Data yang dipakai
Dashboard	Melihat kesehatan model dengan cepat.	Latest TrainingRun, metrik utama, history, per-class.
Eksperimen	Melihat konfigurasi dan artefak training.	TrainingRun + gambar public/training.
Demo Deteksi	Mencoba gambar/video/kamera.	Streamlit online atau FastAPI lokal.
AI Summary	Membaca kesimpulan non-teknis dan rekomendasi.	Metrik database + hasil validasi Ollama/rules.

Keputusan desain

- Sidebar menjaga navigasi konsisten dan menekankan empat kebutuhan assessment.
- Kartu metrik memberi scan cepat; detail tetap tersedia di halaman eksperimen.
- Warna hijau memberi identitas keselamatan/operasional tanpa visual terlalu ramai.
- Layout responsif berubah menjadi satu kolom pada layar kecil.
- Artefak evaluasi dapat dibuka ukuran penuh untuk pembuktian teknis.
- Form admin disembunyikan pada production sehingga recruiter melihat hasil final tanpa risiko mengubah data.

BAB 12

Deployment online dan alasan pemisahannya

Laravel di InfinityFree

InfinityFree menyediakan PHP, MySQL, SSL, subdomain, dan FTP gratis, tetapi tidak menyediakan SSH, Composer, atau NPM. Karena itu seluruh dependency dan asset dibangun lebih dahulu di komputer.

1. `build-package.ps1` menyalin source Laravel yang dibutuhkan.
2. `composer install --no-dev --optimize-autoloader` membuat vendor production.
3. `npm ci` dan `npm run build` membuat `public/build`; `node_modules` lalu dibuang.
4. APP_KEY unik dibuat dan direktori writable disiapkan.
5. Paket awal dan paket `update-safe` dibuat. Update-safe tidak membawa `.env`.
6. Isi `htdocs` diunggah melalui FileZilla.
7. Installer bertoken menjalankan migration + seeder; token lalu dikosongkan agar route kembali 404.

Mengapa root `.htaccess` diperlukan?

Document root InfinityFree adalah `htdocs`, sedangkan entry point Laravel berada di `public/index.php`. Root `.htaccess` menulis ulang request ke `public/`, melarang file tersembunyi seperti `.env`, dan menambahkan security headers.

Streamlit Community Cloud

Streamlit menarik repository GitHub, memasang `streamlit_app/requirements.txt` serta system package dari `packages.txt`, lalu menjalankan `streamlit_app/app.py`. Checkpoint dapat dibundel atau diunduh dari HTTPS dan selalu diverifikasi SHA-256.

Kenapa aplikasi Streamlit kadang “waking up”?

Paket gratis menidurkan aplikasi yang tidak aktif. Ketika dibuka lagi, container, dependency, dan model perlu dimuat. Cold start 1–3 menit bukan error model. Buka link Streamlit langsung, tunggu sampai “Model siap”, lalu kembali ke Laravel.

Port dan endpoint

Komponen	Lokal	Production
Laravel	`127.0.0.1:8000`	`https://ppe-vision.infinityfreeapp.com`
FastAPI YOLO	`127.0.0.1:8001`	Tidak dipakai di InfinityFree
Streamlit	`127.0.0.1:8501`	`*.streamlit.app`
Ollama	`127.0.0.1:11434`	Tidak diekspos ke internet
MySQL	Opsional	Host InfinityFree, port 3306

BAB 13

Keamanan yang sudah diterapkan

Risiko	Kontrol	Lokasi
Kredensial bocor ke Git	`.env` dan Streamlit secrets di-ignore; contoh memakai placeholder.	`.gitignore`
Update menimpa DB password	Paket `update-safe` menghapus `.env`.	`build-package.ps1`
Installer diakses publik	Token minimal panjang, `hash_equals`, throttle; route 404 bila token kosong.	`InstallationController`
Perubahan data oleh pengunjung	`ALLOW_ADMIN_WRITES=false` menyembunyikan route/form tulis di production.	`config/deployment.php`
Upload berbahaya	MIME whitelist, batas ukuran, range confidence, decode gambar aman.	Laravel + FastAPI
Decompression bomb	Batas pixel PIL dan warning diperlakukan sebagai error.	`inference_service/main.py`
Model diganti/rusak	SHA-256 diverifikasi sebelum model dipakai.	`streamlit_app/app.py`
Cross-site request	Form Laravel memakai CSRF; endpoint sensitif diberi throttle.	Blade + routes
Embedding liar	CSP membatasi `frame-src` ke Streamlit dan `frame-ancestors` ke self.	`htdocs.htaccess`
XSS dari AI summary	Markdown direnderer dengan HTML stripped dan unsafe links dinonaktifkan.	`summary/show.blade.php`

Yang tidak boleh dilakukan: mengunggah `.env` lokal ke GitHub, mengirim token/password ke chat, membiarkan installer aktif, menaruh Ollama tanpa autentikasi di internet, atau menonaktifkan validasi upload.

BAB 14

Pengujian, validasi, dan audit

14	75	0
feature tests lulus	assertions lulus	vulnerability Composer/NPM

Apa yang diuji?

- Dashboard menampilkan model dan metrik terbaru.
- Nilai database sama persis dengan `model-info.json`.
- Konfigurasi eksperimen dan impor metrik dapat disimpan saat mode admin aktif.
- Fallback summary bekerja saat Ollama mati.
- Output Ollama disimpan sebagai Markdown terstruktur.
- Artefak evaluasi tampil di detail eksperimen.
- Streamlit tertanam pada halaman deteksi.
- Respons FastAPI dapat disimpan sebagai riwayat deteksi.
- Installer 404 saat token kosong dan menolak token salah.
- Production menyembunyikan operasi admin.

Validasi tambahan

- Model SHA-256 sesuai metadata.
- Composer audit tidak menemukan advisory.
- NPM audit melaporkan 0 vulnerability.
- Build Vite berhasil.
- Dashboard, experiments, summary, detection, Streamlit, dan lock installer telah diperiksa online.

> **Batas makna test:** test software membuktikan integrasi kode, bukan menjamin model sempurna pada seluruh kondisi dunia nyata. Kualitas model tetap harus diuji dengan data lapangan representatif.

BAB 15

File aktif, pendukung, generated, dan opsional

Kategori	Contoh	Tindakan
Aktif production	`app/`, `config/`, `database/`, `resources/`, `routes/`, `public/`, `streamlit_app/`, `best.pt`	Jangan hapus.
Pendukung development	`inference_service/main.py`, `tests/`, README, data JSON/CSV	Pertahankan untuk demo lokal, audit, dan reproduksibilitas.
Generated/ignored	`vendor/`, `node_modules/`, `public/build/`, `deploy/.../package`, `update-safe`, ZIP	Dapat dibuat ulang. Jangan commit; jangan hapus saat sedang dipakai untuk upload.
Konfigurasi rahasia	`.env`, `.streamlit/secrets.toml`	Jangan commit atau bagikan.
Sudah dibersihkan	Konfigurasi Render/Docker dan folder AlwaysData kosong	Dihapus karena bukan bagian deployment final. Dapat dibuat ulang jika arsitektur hosting berubah.

Vendor bukan “sampah”. Folder vendor berisi dependency PHP. Salinannya di package/update-safe memang generated, tetapi diperlukan saat upload karena InfinityFree tidak menjalankan Composer.

BAB 16

Menjalankan keseluruhan sistem secara lokal

A. Laravel

```
composer install
Copy-Item .env.example .env
php artisan key:generate
php artisan migrate --seed
npm install
npm run build
php artisan serve
```

Buka `http://127.0.0.1:8000`.

B. Python environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r inference_service\requirements.txt
python -m pip install -r streamlit_app\requirements.txt
```

C. FastAPI lokal

```
.\.venv\Scripts\python.exe -m uvicorn inference_service.main:app --host 127.0.0.1 --port 8001
```

Isi `YOLO_API_URL=http://127.0.0.1:8001`. Cek `http://127.0.0.1:8001/health`.

D. Streamlit lokal

```
.\.venv\Scripts\python.exe -m streamlit run streamlit_app\app.py
```

Isi `STREAMLIT_DEMO_URL=http://127.0.0.1:8501`.

E. Ollama lokal

```
ollama serve
ollama pull llama3.2:3b
```

Aktifkan tiga variabel Ollama di `.env`, lalu jalankan:

```
php artisan config:clear
```

Urutan start yang mudah diingat

1. Ollama (jika ingin regenerate summary).
2. FastAPI atau Streamlit (pilih kebutuhan demo).
3. Laravel.

4. Buka browser dan uji health/status.

BAB 17

Troubleshooting cepat

Gejala	Penyebab umum	Solusi
HTTP 500 Laravel	DB salah, storage/cache tidak writable, dependency kurang.	Periksa `.env`, log, struktur upload, dan permission.
404 pada semua route	`.htaccess` root/public tidak terunggah.	Upload file bertitik dan refresh FTP listing.
404 pada `/install`	Token kosong.	Normal setelah instalasi; ini kondisi aman.
429 Too Many Requests	Tombol/reload terlalu sering.	Tunggu 1–2 menit; route diberi throttle.
Streamlit “waking up”	Cold start free tier.	Buka link langsung, tunggu 1–3 menit, lalu refresh Laravel sekali.
Model tidak dimuat	`best.pt` tidak ada atau checksum salah.	Pastikan file/path/URL HTTPS dan SHA-256 benar.
Tidak semua 3 kelas tampil	Objek tidak ada atau confidence terlalu tinggi.	Uji input representatif dan threshold 0,20; jangan menganggap kelas model hilang.
Ollama lambat	Cold load model 2 GB dan CPU inference.	Tunggu request pertama; timeout aplikasi sudah 180 detik.
Summary fallback rule-based	Ollama mati, timeout, atau JSON tidak valid.	Periksa `ollama list`, `ollama serve`, URL, dan log Laravel.
Video tidak dapat diputar	FFmpeg tidak tersedia.	Pastikan `packages.txt` memuat `ffmpeg`.

BAB 18

Naskah presentasi 5–7 menit

1. Masalah (30 detik)

PPE Vision membantu mengevaluasi dan mendemokan object detection untuk tiga kelas keselamatan: Helm, Masker, dan People. Targetnya bukan hanya model, tetapi juga sistem yang dapat dipahami dan diakses online.

2. Model dan data (60 detik)

Saya menggunakan checkpoint YOLO26s final. Data dibagi menjadi 3.025 train, 603 validation, dan 212 test independen. Lima random-search trial digunakan untuk memilih hyperparameter, kemudian konfigurasi terbaik dilatih 60 epoch.

3. Hasil (60 detik)

Pada test independen, precision 95,5%, recall 90,7%, F1 93,0%, mAP50 95,7%, dan mAP50-95 69,3%. Kelas terlemah adalah Masker dengan mAP50-95 63,4%, sehingga perbaikan berikutnya fokus pada anotasi dan variasi masker.

4. Arsitektur (75 detik)

Laravel 12 menjadi portal utama dan MySQL menyimpan eksperimen. Streamlit menjalankan Python/YOLO di cloud karena InfinityFree hanya mendukung PHP. Demo Streamlit ditanam ke Laravel melalui iframe. Untuk pengembangan lokal tersedia FastAPI.

5. AI Summary (60 detik)

Ollama llama3.2:3b berjalan lokal. Untuk mencegah halusinasi, LLM tidak mencetak angka; angka selalu diambil dari database. LLM hanya memberi risiko dan tiga rekomendasi dalam JSON schema, kemudian Laravel memvalidasi dan menyediakan fallback rule-based.

6. Keamanan dan deploy (60 detik)

Secret tidak masuk Git, installer bertoken dikunci setelah dipakai, operasi admin dimatikan di production, upload divalidasi, model diverifikasi SHA-256, dan update-safe tidak membawa `.env`.

7. Penutup (30 detik)

Seluruh syarat inti dan point plus terpenuhi. Pengembangan selanjutnya adalah memperbaiki dataset kelas Masker, menambah data lapangan, menguji threshold, dan mengukur performa pada perangkat target.

BAB 19

Pertanyaan wawancara yang mungkin muncul

Q: Mengapa memakai Laravel dan Streamlit sekaligus?

Laravel mengelola UI utama, route, database, validasi, keamanan, dan riwayat eksperimen. Streamlit menjalankan runtime Python/PyTorch yang tidak tersedia di InfinityFree. Pengguna tetap melihat satu portal karena Streamlit di-embed.

Q: Mengapa tidak semuanya dibuat dengan Streamlit?

Syarat point plus meminta Laravel UI/UX. Selain itu Laravel lebih terstruktur untuk CRUD, database, middleware, CSRF, migration, dan halaman laporan.

Q: Mengapa Ollama tidak berjalan langsung di hosting?

Ollama memerlukan proses runtime dan resource model sekitar 2 GB. InfinityFree tidak mendukung proses tersebut. Assessment meminta AI lokal, sehingga Ollama dijalankan lokal dan hasilnya disimpan untuk ditampilkan online.

Q: Bagaimana memastikan AI Summary tidak mengarang angka?

LLM dilarang menulis angka. Laravel menghitung dan mencetak angka dari database. Respons LLM memakai JSON schema, divalidasi, dan memiliki fallback deterministik.

Q: Mengapa memakai Llama 3.2 3B?

Lebih ringan daripada model besar, cukup untuk merangkum metrik dalam Bahasa Indonesia, berjalan lokal tanpa API key, dan latency masih masuk akal pada CPU.

Q: Apa perbedaan validation dan test?

Validation dipantau selama training untuk memilih model. Test independen dipakai setelah keputusan model selesai agar estimasi generalisasi lebih jujur.

Q: Mengapa mAP50-95 dipakai sebagai metrik utama?

Ia menilai kualitas deteksi pada berbagai IoU, sehingga lebih ketat dan representatif daripada hanya mAP50.

Q: Mengapa Masker paling lemah?

Masker berukuran kecil, variasinya besar, mudah tertutup/menyatu dengan wajah, dan kualitas anotasi semuka dapat membuat bounding box tidak konsisten.

Q: Apakah model bisa mendeteksi tiga kelas sekaligus?

Bisa, jika ketiganya ada, terlihat cukup jelas, dan melewati threshold. Model memiliki tiga class head; keluaran per gambar tetap tergantung konten.

Q: Mengapa confidence default 0,20?

Untuk mengurangi objek PPE kecil yang terlewat. Nilai ini trade-off dan harus diuji lagi pada kondisi deployment sebenarnya.

Q: Apa fungsi FastAPI bila sudah ada Streamlit?

FastAPI adalah jalur integrasi lokal Laravel → Python yang menyimpan riwayat deteksi ke DB. Streamlit adalah demo mandiri untuk hosting online dan video/kamera.

Q: Bagaimana model dijaga integritasnya?

Metadata menyimpan SHA-256. Streamlit menghitung checksum sebelum memuat model; file yang tidak cocok ditolak.

Q: Bagaimana upload diamankan?

MIME/ekstensi dibatasi, ukuran dibatasi, image didecode, pixel maksimum ditetapkan, dan threshold divalidasi.

Q: Mengapa installer lewat web?

InfinityFree gratis tidak memberi SSH. Installer bertoken menjadi jembatan menjalankan migration dan seeder; setelah selesai token dikosongkan sehingga route 404.

Q: Bagaimana update tanpa merusak konfigurasi?

Upload isi `update-safe/htdocs`; paket ini sengaja tidak memiliki `.env`, sehingga APP_KEY dan kredensial MySQL server tidak tertimpa.

Q: Mengapa Streamlit lambat saat pertama dibuka?

Free tier menidurkan aplikasi. Container perlu bangun dan model perlu dimuat. Request berikutnya lebih cepat karena resource cache.

Q: Apa kekurangan sistem saat ini?

Masker masih kelas terlemah, demo gratis memiliki cold start, inferensi video memakai CPU, belum ada tracking identitas unik antar-frame, dan evaluasi lapangan perlu diperluas.

Q: Apa pengembangan selanjutnya?

Audit anotasi Masker, tambah hard examples, lakukan class balancing/augmentation terkontrol, threshold calibration, tracking video, monitoring latency, dan uji pada data lokasi target.

Q: Apakah angka dashboard dan summary benar-benar sama?

Ya. Feature test membandingkan seed database terhadap `model-info.json` hingga toleransi 1e-9 dan menguji angka yang dirender di dashboard.

Q: Apa yang terjadi bila Ollama gagal?

Exception dicatat, kemudian sistem memakai summary lokal berbasis aturan. Halaman tetap berfungsi dan engine ditandai bukan LLM.

BAB 20

Glosarium, checklist, dan link final

Glosarium singkat

Bounding box	Kotak koordinat yang mengelilingi objek.	**Checkpoint**	File bobot model hasil training, yaitu `best.pt`.
Inference	Proses memakai model untuk memprediksi input baru.	**Epoch**	Satu putaran model melihat seluruh data train.
False positive	Model mendeteksi objek yang sebenarnya tidak ada/salah.	**False negative**	Objek ada tetapi tidak terdeteksi.
Cold start	Waktu menyalakan kembali layanan/model setelah tidur.	**Seeder**	Kode untuk mengisi data awal/final ke database.
Migration	Versi struktur tabel database.	**Iframe**	Menampilkan halaman layanan lain di dalam halaman Laravel.

Checklist sebelum presentasi

- ✓ Buka dashboard dan tunjukkan test independen.
- ✓ Buka detail eksperimen dan artefak asli.
- ✓ Bangunkan Streamlit 3 menit sebelum presentasi.
- ✓ Siapkan satu gambar yang memuat orang, helm, dan masker.
- ✓ Buka AI Summary dan jelaskan kontrol halusinasi.
- ✓ Jangan menampilkan `.env`, password FTP/MySQL, atau token.
- ✓ Ingat limitation dan rencana perbaikan Masker.

Link final

Portal utama	https://ppe-vision.infinityfreeapp.com/
Eksperimen	https://ppe-vision.infinityfreeapp.com/experiments
AI Summary	https://ppe-vision.infinityfreeapp.com/experiments/1/summary
Demo dalam Laravel	https://ppe-vision.infinityfreeapp.com/detection
Streamlit langsung	https://ppe-vision-lculpeehh7bzalqxpwtbf5.streamlit.app/
GitHub	https://github.com/silviadela/ppe-vision

> **Satu link yang diberikan kepada recruiter:** https://ppe-vision.infinityfreeapp.com/. Link lain adalah bukti tambahan bila diminta.

Laporan dibuat dari audit source code lokal dan deployment aktif pada 15 Oktober 2026. Tidak ada password, token, APP_KEY, atau kredensial database yang disertakan.